R080 Timer MEGA — Documento Tecnico

Documento di riferimento per chi vuole capire il firmware nei dettagli, senza giri di parole. Tutto quello che serve per leggere, modificare, debuggare e non rompere il timer.

> Versione: branch `Libraries_Refactoring` — 2026-05-24 Target: Arduino Mega 2560 (ATmega2560 @ 16 MHz) Framework: PlatformIO + Arduino Core

---

1. Cos'è R080 Timer MEGA

È il firmware di un **timer da camera oscura** per ingranditore fotografico.

Quando stampi una foto a contatto o a ingrandimento, devi tenere la lampada dell'ingranditore accesa per un certo tempo (di solito tra 1 secondo e qualche decina di secondi). Questo timer:

1. Comanda un **relè** che accende/spegne la lampada dell'ingranditore.
2. Mostra il tempo su un **display a 7 segmenti** (es. `10.0` = 10 secondi).
3. Mostra modalità, precisione, canale su un **LCD 16x2**.
4. Riceve input dall'utente con **keypad 4x4 + 2 encoder + 1 pulsante principale**.
5. Salva 3 "ricette" (canali A/B/C) in **EEPROM** così non si perdono allo spegnimento.

Ha 3 modalità di lavoro:

- **FOCUS** — la lampada resta accesa per messa a fuoco, il timer conta in salita.
- **COUNTDOWN** — la lampada si accende per N decimi di secondo poi si spegne da sola.
- **TEST STRIP** — esposizioni a gradini per provare diversi tempi sulla stessa striscia di carta.

---

2. Hardware

2.1 La scheda

- **MCU**: ATmega2560 (16 MHz, 8 KB RAM, 256 KB Flash, 4 KB EEPROM).
- **Alimentazione**: 5 V via USB o jack DC.
- **Carico esterno**: relè a stato solido o relè meccanico, comandato 5 V.

2.2 Periferiche e pin

Definiti tutti in `include/constants.h`. Riassunto:

Periferica	Pin Arduino	Note
Buzzer	D49	`tone()` standard, frequenze 300/600 Hz
Relè ingranditore	D47	PORTL bit PL2 (dettagli sotto)
Pulsante principale	D2	INPUT_PULLUP, gestito da OneButton
Display 7-seg (TX seriale)	A2	SoftwareSerial @ 9600 baud (SparkFun)
LCD I²C Waveshare	SDA=D20, SCL=D21	Indirizzi 0x3E (LCD) + 0x60 (RGB)
Encoder modalità (DT/CLK)	D3 / D4	Rotazione: cicla FOCUS/COUNTDOWN/TEST
Pulsante encoder modalità	D17	Toggle modalità continua/step-by-step
Encoder precisione (DT/CLK)	D18 / D19	Rotazione: cicla 11 precisioni
Pulsante encoder precisione	D16	Toggle SECONDS/FSTOP
Keypad colonne	D41, D39, D37, D35	Matrice 4x4
Keypad righe	D33, D31, D29, D27	

2.3 Il keypad

```
1   2   3   A
4   5   6   B
7   8   9   C
*   0   #   D
```

- `0..9` — inserimento cifre (max 4 = `9999` decimi = 999.9 s)
- `A`, `B`, `C` — seleziona canale 1, 2, 3 (rilascio)
- `D` — reset canale corrente (rilascio)
- `*` — diminuisci / step F-stop indietro / brightness -1 (FOCUS)
- `#` — aumenta / step F-stop avanti / brightness +1 (FOCUS)
- `*` (HOLD) — toggle mute audio
- `#` (HOLD) — toggle relè per il canale corrente

---

3. L'architettura del software in 5 minuti

3.1 Single Translation Unit (TU)

Tutto il firmware è UN SOLO file .cpp (`src/main.cpp`). Sotto `include/functions/*.h` ci sono "header" che però **definiscono** le funzioni (non le dichiarano soltanto). `main.cpp` li include in un ordine preciso.

Perché?

- Su AVR il compilatore inlina meglio e ottimizza di più se vede tutto in una volta.
- Niente prototipi `extern` da gestire.
- Niente linker errors strani.

Il prezzo:

- Non puoi aggiungere un secondo `.cpp` che includa `variables.h` → multiple definition.

- L'ordine degli `#include` è importante.

3.2 L'ordine degli include in `main.cpp`

```
#include "types.h"             // enum, struct, classi base
#include "constants.h"         // pin, toni, stringhe
#include "variables.h"         // tutte le variabili globali
#include "functions/timer_isr.h"   // ISR + macro relè   <- DEVE essere prima di base.h
#include "functions/base.h"        // usa RELAY_ON/OFF
#include "functions/config.h"      // init hardware
#include "functions/presentation.h"// rendering LCD/7-seg
#include "functions/sound.h"       // suoni non-bloccanti
#include "functions/storage.h"     // EEPROM
#include "functions/running_mgmt.h"// loop main button
#include "functions/operations.h"  // 3 modalità
#include "functions/encoder_mgmt.h"// encoder
#include "functions/keypad_mgmt.h" // keypad
#include "functions/timer_manager.h"// start/stop timer + handler ISR
```

Cambiare l'ordine = compilazione che esplode.

3.3 Le 3 lingue del codice

Nel codice convivono 3 "dialetti":

1. **Arduino classico** — `digitalWrite`, `pinMode`, `tone()`, `millis()`.
2. **AVR raw** — manipolazione diretta dei registri (PORTL, TIMER1, OCR1A, ATOMIC_BLOCK).
3. **C++** — `class`, `struct`, function pointer, costruttori (per `Operation`).

Si usa AVR raw solo dove serve la velocità o l'atomicità (ISR e relè).

---

4. Il motore temporale — Timer1 ISR @ 1 kHz

Questa è la parte più "magica" del firmware. Vale la pena capirla bene.

4.1 Cos'è una ISR

Una **Interrupt Service Routine** è una funzione che viene chiamata automaticamente dall'hardware (non dal codice) quando succede un evento. Nel nostro caso, l'evento è "il timer hardware ha raggiunto un certo valore".

Il chip ha un timer hardware (Timer1) che conta da solo, indipendente dal main loop. Quando arriva al numero giusto, **sospende quello che il main sta facendo, esegue la ISR, e torna**.

4.2 La configurazione

In `timer_isr.h` → `setupTimer1()`:

```
OCR1A  = 249;                          // 250 tick * 4 us = 1 ms
TCCR1B |= (1 << WGM12);                // modalità CTC
TCCR1B |= (1 << CS11) | (1 << CS10);   // prescaler 64
TIMSK1 |= (1 << OCIE1A);               // abilita interrupt
```

Conti veloci:

- Clock CPU = 16 MHz → 16 000 000 cicli al secondo.
- Prescaler 64 → il timer avanza ogni 64 cicli = 4 µs.
- OCR1A = 249 → l'interrupt scatta ogni 250 tick = 1000 µs = **1 ms**.
- 1000 interrupt al secondo = 1 kHz.

4.3 Cosa fa la ISR ad ogni millisecondo

```
ISR(TIMER1_COMPA_vect) {
  g_tickMs++;                  // contatore globale in ms (sempre attivo)

  switch (g_modeActive) {
    case 2: // COUNTDOWN
      if (g_remainingMs > 0) {
        g_remainingMs--;
        if (g_remainingMs == 0) {
          if (g_relayGate) RELAY_OFF();   // spegne lampada al ms preciso
          g_endOfCountdown = 1;            // alza flag per il main
        } else if ((g_remainingMs % 1000) == 0) {
          g_metronomeFlag = 1;             // tick al secondo
        }
      }
      break;
    case 3: // TEST_STRIP
      g_elapsedMs++;
      if (g_elapsedMs >= g_nextStepTargetMs) {
        g_nextStepReached = 1;
      }
      break;
  }

  if (++g_100Counter >= 100) {
    g_100Counter = 0;
    g_tickFlag100ms = 1;        // cadenza rendering 10 Hz
  }
}
```

In pratica la ISR:

- Aggiorna `g_tickMs` (orologio globale del firmware).
- Decrementa il countdown e apre il relè quando arriva a zero.
- Conta l'elapsed del test strip e alza un flag quando si raggiunge il target.
- Ogni 100 ms alza `g_tickFlag100ms` (per il rendering).
- Ogni 1000 ms in countdown alza `g_metronomeFlag` (per il beep al secondo).

4.4 Perché ISR e non `millis()`?

Prima della rifattorizzazione c'era `arduino-timer` che usava `millis()` + callback chiamati dal main loop. Problemi:

- **Jitter**: se il main loop è impegnato a fare altro (LCD lento), il countdown salta.
- **Precisione**: il relè si spegneva con +/- 10 ms di errore.
- **Doppio motore temporale**: Timer0 (millis) + scheduler nostro = drift.

Con la ISR:

- Il countdown si decrementa esattamente ogni ms.
- Il relè si apre **al microsecondo giusto** dal punto di vista del main loop.
- Il main loop fa solo rendering e input — può anche essere lento, non importa.

4.5 Le variabili condivise ISR ↔ main

Sono dichiarate `volatile` in `variables.h`:

```
volatile uint32_t g_tickMs;           // orologio globale (ms dal boot)
volatile uint32_t g_remainingMs;      // residuo countdown
volatile uint32_t g_elapsedMs;        // elapsed test-strip
volatile uint32_t g_nextStepTargetMs; // target prossimo step
volatile uint8_t  g_relayGate;        // snapshot canale all'armo
volatile uint8_t  g_modeActive;       // 0=none, 1=focus, 2=countdown, 3=teststrip
volatile uint8_t  g_tickFlag100ms;    // flag rendering 10 Hz
volatile uint8_t  g_endOfCountdown;   // flag fine countdown
volatile uint8_t  g_nextStepReached;  // flag step raggiunto
volatile uint8_t  g_metronomeFlag;    // flag tick al secondo
```

4.6 ATOMIC_BLOCK: perché serve

Quando il main loop legge `g_remainingMs` (`uint32_t` = 4 byte), AVR non può leggere 4 byte in un colpo solo. Li legge un byte alla volta. **Se la ISR scatta tra il byte 1 e il byte 2**, può modificare la variabile, e il main legge un valore "spezzato" (es. half-old, half-new).

Soluzione: disabilitare gli interrupt durante la lettura.

```
uint32_t snapshot;
ATOMIC_BLOCK(ATOMIC_RESTORESTATE) {
  snapshot = g_remainingMs;   // 4 byte letti senza interruzioni
}
// usa snapshot, gli interrupt ora possono scattare di nuovo
```

`ATOMIC_RESTORESTATE` = "rimetti lo stato SREG.I com'era prima del blocco" (non riabilita se erano disabilitati).

Regola: ogni accesso main-side a una `volatile uint32_t` o a flag che vogliamo leggere+azzerare in modo coerente DEVE essere in `ATOMIC_BLOCK`. Sui `volatile uint8_t` la lettura singola è già atomica (1 byte = 1 istruzione), ma se la accoppi a una scrittura (leggi-azzera-fa-qualcosa) serve comunque ATOMIC.

---

5. Il loop principale

Da `src/main.cpp`:

```
void loop() {
  mainButton.tick();   // OneButton: gestisce debounce, click, longpress

  uint8_t flag100, flagEnd, flagStep, flagMetro;
  ATOMIC_BLOCK(ATOMIC_RESTORESTATE) {
    flag100   = g_tickFlag100ms;   g_tickFlag100ms   = 0;
    flagEnd   = g_endOfCountdown;  g_endOfCountdown  = 0;
    flagStep  = g_nextStepReached; g_nextStepReached = 0;
    flagMetro = g_metronomeFlag;   g_metronomeFlag   = 0;
  }

  if (flagEnd)   onCountdownEnd();
  if (flagStep)  onTestStripStep();
  if (flagMetro) onMetronomeBeat();
  if (flag100)   onTick100ms();

  tickSound();              // SoundPlayer state machine
  manageMainButtonAction(); // input handling (solo se timer fermo)
}
```

Cosa succede a ogni giro (tipico 1000+ giri/s):

1. `mainButton.tick()` — OneButton legge il pulsante.
2. Snapshot dei flag della ISR (in modo atomico).
3. Se `flagEnd` è alto → il countdown è finito, fai cleanup.
4. Se `flagStep` è alto → step beep nel test strip, calcola prossimo target.
5. Se `flagMetro` è alto → emetti tick del metronomo.
6. Se `flag100` è alto → ridisegna LCD/7-seg (10 volte al secondo basta).
7. `tickSound()` — fa avanzare la macchina a stati del suono.
8. `manageMainButtonAction()`:
    - Se il timer è **fermo** → leggi keypad + encoder.
    - Se il timer è **attivo** → ri-arma il timer (no-op se già armato).

Importante: terminal events (`onCountdownEnd`, `onTestStripStep`, `onMetronomeBeat`) sono processati **prima** di `onTick100ms`. Così il rendering vede lo stato pulito post-evento.

---

6. Le 3 modalità

6.1 FOCUS — l'ingranditore acceso per la messa a fuoco

Comportamento:

- Si avvia con il main button.
- Il relè va ON e resta ON.
- Il timer conta in salita da 0 (in decimi di secondo).
- Si ferma solo quando l'utente ripreme il main button.
- `*` / `#` nel keypad regolano la luminosità del display 7-seg (1..10).

Dietro le quinte:

- `g_modeActive = 1`.

- L'ISR aggiorna solo `g_tickMs` e cadenza 100 ms. **Non incrementa nessun contatore** per la modalità FOCUS.
- L'incremento di `timerCurrentValue` avviene **main-side** in `onTick100ms` (`timerCurrentValue++` ogni 100 ms).
- Niente precisione, niente modalità, niente F-stop in FOCUS.

Perché incrementare main-side e non in ISR?

- FOCUS ha precisione 100 ms (un decimo di secondo).
- Non c'è motivo di sprecare cicli ISR per qualcosa che il main loop fa già a 10 Hz.
- Mantiene la ISR minimale.

6.2 COUNTDOWN — esposizione programmata

Comportamento:

- Imposti il tempo (es. `123` → `12.3` secondi).
- Premi main button.
- Il relè si accende, il timer conta in discesa.
- Ogni secondo c'è un beep di metronomo (tono basso 300 Hz).
- Al raggiungimento di zero: relè OFF + beep finale.
- Il tempo si "ricarica" automaticamente (repeat-last-exposure) per la prossima stampa.

Dietro le quinte:

All'avvio (`startCountdownTimer`):

```
if (appoTimeSet <= 0) return;       // guard: tempo 0 abortisce
timerCurrentValue = appoTimeSet;     // copia il tempo backup
relayStart(true);                    // accende il relè (se canale attivo)
isActionStarted = true;
showTime(timerCurrentValue);
ATOMIC_BLOCK(ATOMIC_RESTORESTATE) {
  g_remainingMs = (uint32_t)appoTimeSet * 100UL;  // decimi -> ms
  g_elapsedMs = 0;
  g_relayGate = channelsRelayActivated[channel - 1];  // snapshot canale
  g_modeActive = 2;
}
```

Da qui in poi la ISR fa tutto:

- Decrementa `g_remainingMs` di 1 ogni ms.
- A `g_remainingMs == 0` : `RELAY_OFF()` + alza `g_endOfCountdown` .
- Ogni 1000 ms (cioè a 999, 1999, 2999, ...): alza `g_metronomeFlag` .

Il main loop:

- Vede `g_metronomeFlag = 1` → chiama `onMetronomeBeat()` → emette beep 300 Hz × 80 ms.
- Vede `g_endOfCountdown = 1` → chiama `onCountdownEnd()` :

```
isActionStarted = false;
isMainBtnPressed = false;
emitStopSound();             // sequenza beep di stop
timeSet = appoTimeSet;       // ripristina il tempo per il prossimo shot
showTime(0);
```

- Ogni 100 ms (`onTick100ms`): legge `g_remainingMs`, lo divide per 100, mostra sul 7-seg.

Repeat-last-exposure = `timeSet = appoTimeSet` al termine. Significa: se hai impostato 12.3 s e premi start, dopo che è finito il display torna a `12.3` (non a `0.0`). Pratico per stampe in serie.

Perché `appoTimeSet` esiste? Perché `timeSet` viene resettato a 0 a fine countdown. `appoTimeSet` (= backup) preserva il valore originale.

6.3 TEST STRIP — esposizioni a gradini

Comportamento:

- Hai una striscia di carta. Vuoi provare 4 tempi: 5 s, 7 s, 10 s, 14 s (gradini F-stop di 1/2).
- Copri tutta la striscia tranne il primo pezzo, premi start.
- Il timer conta in salita, suona un beep ad ogni "step" (5 s, 7 s, 10 s, 14 s).
- Tra uno step e l'altro tu sposti il cartoncino per scoprire il pezzo successivo.

Ci sono **2 modalità interne**:

- **CONTINOUS** (sì, scritto male nel codice di proposito) — il timer continua a girare, il relè resta sempre ON.
- **STEP_BY_STEP** — ad ogni step il relè si spegne e bisogna ripremere main button per il prossimo.

Dietro le quinte:

All'avvio (`startTestStripTimer`):

```
ATOMIC_BLOCK(ATOMIC_RESTORESTATE) { elapsedSnap = g_elapsedMs; }
if (elapsedSnap == 0) {
  // fresh start
  nextStepTarget = appoTimeSet > 0 ? appoTimeSet : 10;
  timerCurrentValue = 0;
  ATOMIC_BLOCK(ATOMIC_RESTORESTATE) {
    g_elapsedMs = 0;
    g_nextStepTargetMs = (uint32_t)nextStepTarget * 100UL;
  }
}
// se elapsedSnap > 0 siamo in resume STEP_BY_STEP: g_elapsedMs e
// g_nextStepTargetMs sono già impostati, non toccare nulla
relayStart(true);
isActionStarted = true;
ATOMIC_BLOCK(ATOMIC_RESTORESTATE) {
  g_relayGate = channelsRelayActivated[channel - 1];
  g_modeActive = 3;
}
```

ISR:

- Incrementa `g_elapsedMs` ogni ms.
- Quando `g_elapsedMs >= g_nextStepTargetMs` → alza `g_nextStepReached`.

Main loop (`onTestStripStep`):

- Emette beep di step (600 Hz × 150 ms).
- Calcola il prossimo target:
  - Se precisione **FSTOP**: `newTarget = round(timerCurrentDecis * 2^(1/precisionValue))` (almeno +1).
  - Se precisione **SECONDS**: `newTarget = timerCurrentDecis + precisionValue * 10`.
- Aggiorna `g_nextStepTargetMs` atomicamente.
- Se modalità **STEP_BY_STEP**: spegne relè, ferma il conteggio (`g_modeActive = 0`).
- Se modalità **CONTINOUS**: continua a girare.

Esempio numerico (FSTOP precisione 1/2, base 5 s):

- t=0 → start, target = 5.0 s
- t=5 s → beep, calcolo nuovo target: `round(50 * 2^(1/2)) = round(50 * 1.414) = round(70.7) = 71` decimi = 7.1 s
- t=7.1 s → beep, calcolo: `round(71 * 1.414) = 100` decimi = 10.0 s
- t=10.0 s → beep, calcolo: `round(100 * 1.414) = 141` decimi = 14.1 s
- ...

Warning beep: in `onTick100ms`, se la distanza dal prossimo step è ≤ 3 decimi (0.3 s), emette un beep corto di avviso (600 Hz × 50 ms). Suona 3 volte (a 0.3, 0.2, 0.1 s dallo step) per dare il tempo all'utente di prepararsi.

---

7. F-Stop — la matematica del fotografo

7.1 Cos'è uno "stop"

In fotografia, **+1 stop = il doppio della luce, +2 stop = quadruplo, ecc.** È una scala logaritmica in base 2.

Per il tempo di esposizione:

- +1 stop = raddoppio il tempo (es. 5 s → 10 s)
- -1 stop = dimezzo il tempo (es. 5 s → 2.5 s)
- +1/2 stop = `5 * 2^(1/2) = 7.07 s`
- +1/3 stop = `5 * 2^(1/3) = 6.30 s`

7.2 La struttura `FStopState`

Da `fstop.h`:

```
struct FStopState {
    int baseTime;     // tempo base in decimi (es. 50 = 5.0 s)
    int stopOffset;   // offset in step (es. +2, -1)

    int getCalculatedTime(int precisionValue) const {
        if (baseTime <= 0 || precisionValue <= 0) return baseTime;
        float multiplier = pow(2.0, (float)stopOffset / (float)precisionValue);
        int result = (int)((float)baseTime * multiplier + 0.5f);
        return constrain(result, 1, 9999);
    }
};
```

`precisionValue` è il **denominatore**: 1 = full stop, 2 = 1/2 stop, 3 = 1/3 stop, ecc. Il timer offre 11 precisioni F-stop: `1, 2, 3, 4, 6, 8, 12, 16, 24, 32, 64`.

7.3 Quando il `baseTime` si aggiorna

Questo è il punto delicato. Il `baseTime` è il tempo "origine" da cui si calcolano gli offset.

Si aggiorna in 4 punti:

1. **Ogni cifra del keypad** (`evaluateKey` case digit) → `fstopState.setBase(timeSet)`.
2. **Switch canale** (`switchToChannel`) → `setBase(timeSet)` del canale appena caricato.
3. **Switch precisione SECONDS→FSTOP** (`onPrecisionButtonEvent`) → `setBase(timeSet > 0 ? timeSet : 1)`.
4. **`*` / `#` in F-stop con base=0** (lazy init) → `if (baseTime <= 0) setBase(...)`.

Quando si aggiorna il base, `stopOffset` viene resettato a 0.

7.4 Esempio concreto F-stop

Setup: canale A, precisione F-stop 1/3, tempo 5.0 s.

```
baseTime=50, stopOffset=0
Display: 5.0 (offset " 00")
```

Utente preme `#` (step up):

```
baseTime=50, stopOffset=+1
getCalculatedTime(3) = 50 * 2^(1/3) = 50 * 1.260 = 63.0 ≈ 63
timeSet = 63
Display: 6.3 (offset "+01")
```

Utente preme `#` di nuovo:

```
baseTime=50, stopOffset=+2
getCalculatedTime(3) = 50 * 2^(2/3) = 50 * 1.587 = 79.4 ≈ 79
timeSet = 79
Display: 7.9 (offset "+02")
```

Utente preme `*` (step down):

```
baseTime=50, stopOffset=+1
getCalculatedTime(3) = 63
Display: 6.3 (offset "+01")
```

Utente cambia precisione a 1/2:

```
evaluateEncoderPrec ricalcola:
baseTime=50, stopOffset=+1, precisionValue=2
getCalculatedTime(2) = 50 * 2^(1/2) = 50 * 1.414 = 70.7 ≈ 71
timeSet = 71
Display: 7.1 (offset "+01", ora però il significato è "+1/2 stop")
```

7.5 Layout dell'offset sul display LCD

Da `presentation.h:85-93`:

- Posizione: riga 1, colonne 5-7 (3 caratteri).
- Formato: `+02` per positivi, `-01` per negativi, `00` per zero.
- Solo in modalità F-stop. In SECONDS le 3 colonne sono pulite.

---

8. I canali e la memoria EEPROM

8.1 I 3 canali

Pensali come "preset". Ogni canale ha:

- Un tempo (`channelTime[0..2]`, in decimi).
- Un flag relè abilitato (`channelsRelayActivated[0..2]`).

L'utente preme `A` / `B` / `C` per cambiare canale. Il tempo corrente viene salvato sul canale di partenza, il nuovo canale viene caricato.

```
void switchToChannel(byte ch) {
    channelTime[channel - 1] = timeSet;    // salva canale uscente
    channel = ch;
    timeSet = channelTime[channel - 1];     // carica canale entrante
    appoTimeSet = timeSet;
    nCifra = 0;
    fstopState.setBase(timeSet);            // reset F-stop offset
    showChannel();
    showTime(timeSet);
    saveSettings();                          // persiste su EEPROM
}
```

> **Indicizzazione:** `channel` è 1, 2, o 3 (mai 0). Gli array sono 0-indexed. Quindi `channelTime[channel-1]`. Sembra una banalità ma ci sono stati off-by-one nel passato (vedi "Bug fixati" sotto).

8.2 Il flag relè per canale

Tieni premuto `#` per togglare. Cosa fa?

Spegne (o riaccende) il **relè fisico** per quel canale, **lasciando girare il timer normalmente**. Caso d'uso: vuoi simulare un'esposizione senza accendere davvero la lampada (es. per provare la sequenza sonora del test strip).

Il check è in `relayStart` (`base.h`):

```
void relayStart(boolean start) {
    if (channelsRelayActivated[channel - 1]) {
      if (start) { RELAY_ON(); } else { RELAY_OFF(); }
    }
}
```

Se il flag è false, le macro `RELAY_ON()` / `RELAY_OFF()` non vengono mai chiamate dal main. Ma attenzione: la **ISR** all'apertura di countdown usa `g_relayGate` (snapshot del flag), non il flag corrente. Quindi il comportamento è coerente anche se l'utente toglie il flag a metà countdown (cosa che comunque non può fare, perché il keypad è disabilitato durante il countdown).

8.3 EEPROM layout

13 byte totali, partendo dall'indirizzo 0. Da `storage.h`:

Addr	Bytes	Campo
0	1	Magic byte `0xA5`
1	1	Brightness (1..10)
2	1	Mute (0 o 1)
3	1	Canale attivo (1..3)
4-5	2	`channelTime[0]` (int16_t)
6-7	2	`channelTime[1]`
8-9	2	`channelTime[2]`
10	1	`channelsRelayActivated[0]`
11	1	`channelsRelayActivated[1]`
12	1	`channelsRelayActivated[2]`

Il magic byte 0xA5 serve a distinguere "EEPROM vuoto" da "EEPROM valido". Se il primo byte non è `0xA5`, `loadSettings()` ritorna `false` e si usano i default.

8.4 `eeprom_update_*` vs `eeprom_write_*`

L'EEPROM ha circa **100 000 cicli di scrittura** per byte. Se scriviamo ad ogni save, lo bruciamo in fretta.

`eeprom_update_byte/word`:

- Legge prima.
- Se il valore è già giusto, NON scrive.

- Scrive solo se è cambiato.

Risultato: la maggior parte dei `saveSettings()` non tocca davvero la EEPROM.

---

9. Input — keypad, encoder, main button

9.1 La logica generale

L'input viene processato **solo quando il timer è fermo**:

```
void manageMainButtonAction() {
  !isMainBtnPressed ? onStop() : manageStarted();
}

void onStop() {
  stopAllTimers();
  evaluateEncoderMod();   // legge encoder modalità
  evaluateEncoderPrec();  // legge encoder precisione
  evaluateButtons();      // bottoni encoder
  evaluateKey();          // keypad
}
```

Quando `isMainBtnPressed == true` (timer attivo), gli input vengono **completamente ignorati**. Questo è intenzionale: evita che l'utente cambi modalità a metà countdown.

9.2 Il keypad

Usa la libreria `Keypad` di Chris-A. È una state machine: ogni tasto può essere in `IDLE`, `PRESSED`, `HOLD`, `RELEASED`.

Da `keypad_mgmt.h`:

```
switch (keypad.key[ki].kstate) {
    case HOLD: {
        // toggle mute, toggle relè canale
        // !! NIENTE break — cade in PRESSED/IDLE
    }
    case PRESSED:
    case IDLE:
        return;  // ignora questi stati

    case RELEASED: {
        // qui le azioni vere
    }
}
```

Quirk noto: il `HOLD` non ha `break`. È **intenzionale**. Se aggiungessimo `break` cadrebbe nel `case RELEASED` quando il tasto viene rilasciato dopo HOLD, eseguendo due volte l'azione (HOLD + RELEASED). Lasciando cadere in `PRESSED/IDLE: return`, si esce subito e si evita questo. Da NON "correggere".

Altro quirk: il `MATRIX_KEYS` è dichiarato `[COLS_N][ROWS_N]` ma `makeKeymap` viene chiamato con `ROWS_PIN` prima. I due swap si cancellano in pratica. Storia: bug di copy-paste lontano, ora consolidato. NON correggere.

9.3 Inserimento numerico

`nCifra` conta le cifre inserite (0..3, max 4 cifre):

```
case 0: newTimeSet = digitValue;              // prima cifra: rimpiazza
case 1..3: newTimeSet = timeSet * 10 + digit; // cifre successive: appendi
```

Esempio: utente preme `1`, `2`, `3` → `timeSet = 123` decimi = `12.3` s. Quarta cifra (`9`) → `1239` decimi = `123.9` s. Quinta cifra ignorata (max 4).

9.4 Gli encoder

Usa la libreria `Encoder` di Paul Stoffregen. Ogni "scatto" del manico fisico produce 4 conteggi nel chip, quindi si fa `read() >> 2`.

```
long newEncoderPosition = encoderMode.read() >> 2;
int timerModeIndex = (newEncoderPosition % 3 + 3) % 3;
```

Il `(x % 3 + 3) % 3` è il classico trucco per avere un modulo positivo anche con numeri negativi (rotazione antioraria).

9.5 Il main button

Usa `OneButton` con questi parametri:

```
mainButton.setDebounceMs(50);
mainButton.setClickMs(100);
mainButton.setPressMs(1000);
```

Su click: toggla `isMainBtnPressed`. È **volatile** perché la callback è invocata da un altro flusso (`mainButton.tick()` chiamato dal loop).

---

10. Output — LCD, 7-seg, buzzer

10.1 Il display LCD 16x2

È un display Waveshare RGB I²C. Driver in `lib/WaveShare-Lib/`.

- Indirizzo LCD: 0x3E.
- Indirizzo RGB backlight: 0x60.
- Hardcoded nel driver, l'`LCD_ADDRESS = 0x27` in `constants.h` è residuo storico, non usato.

Colore retroilluminazione fissato **rosso** in `configLCD()`:

```
lcd.setRGB(255, 0, 0);
```

Scelta UX: il rosso è la luce di sicurezza in camera oscura e non impressiona la carta fotografica.

Layout (16 colonne × 2 righe):

Riga 0 (modalità + precisione + modalità test strip):

```
Posizione: 0123456789012345
FOCUS:     FOCUS
COUNTDOWN: COUNTDOWN  [ ] S
TESTSTRIP: TEST STRIP[ ] SCO
                       ^ ^ ^
                       12 14
```

Colonna 12: `S` (SECONDS) o `F` (FSTOP). Colonna 14-15: `CO` (continuo) o `PP` (step-by-step) — solo TEST STRIP.

Riga 1 (precisione + canale + icone):

```
Posizione: 0123456789012345
FSTOP:     Pr:1/4+02 Ch:A ♪⚡
SECONDS:   Pr: 5s    Ch:A ♪⚡
FOCUS:               Ch:A ♪⚡
```

- Cols 0-7: stringa precisione (8 char, PROGMEM).
- Cols 5-7: offset F-stop (sovrascrive in FSTOP).
- Cols 9-12: canale `Ch:A`.
- Col 14: nota musicale (♪) se audio attivo.
- Col 15: spina (⚡) se relè canale abilitato.

Caratteri custom:

- Slot 1 (♪): `note[8] = { 0x2, 0x3, 0x2, 0x2, 0xe, 0x1e, 0xc, 0x0 }`.
- Slot 2 (⚡): `spina[8] = { 0xa, 0xa, 0x1f, 0x11, 0x11, 0xe, 0x4, 0x4 }`.

10.2 Il display 7-segmenti

È un SparkFun Serial 7-Seg (o cloni). Comunica via UART a 9600 baud.

Usa `SoftwareSerial(1, A2)` perché `Serial` (hardware) è già usato per debug USB. **A2 è TX**, pin 1 è "RX dummy" — non riceviamo dati dal display.

> **Nota tecnica:** pin 1 è anche il TX hardware di `Serial`. Funziona per due ragioni fortuite: (1) l'USART quando ha `TXEN0=1` sovrascrive il DDR del pin, quindi `Serial.println(...)` esce comunque; (2) pin 1 (PE1) non ha PCINT sul Mega 2560, quindi `SoftwareSerial::setRX(1)` non registra un interrupt. È un design fragile ma funzionante.

Comandi inviati al display:

- `0x76` — reset
- `0x77 0bXXXXXXXX` — controllo punto decimale (bit per cifra)
- `0x7A <0-255>` — brightness
- `"NNNN"` — 4 cifre ASCII

Esempio `showTime(123)`:

```
sevenSegmentsDisplay.write(0x77);
sevenSegmentsDisplay.write(0b00000100);  // dp dopo cifra 1
sprintf(keyBuffer, "%04i", 123);          // "0123"
sevenSegmentsDisplay.write(keyBuffer);    // mostra "0.123" → "12.3" letto
```

Attenzione al `0b00000100`: il bit settato è la posizione del punto decimale tra le 4 cifre. Cambiandolo si rompe la lettura del tempo.

10.3 Il buzzer e i suoni

`tone(BUZZER_PIN, freq, duration)` è già **non-bloccante**: avvia il suono e ritorna subito. AVR usa Timer2 per generarlo.

2 frequenze:

- `TONE_UP = 600` Hz (conferme, step beep, warning, unmute)
- `TONE_DOWN = 300` Hz (metronomo, stop)

Sequenze sonore: quando devo emettere più beep in fila (es. la sequenza di stop), uso una macchina a stati in `sound.h`:

```
struct SoundPlayer {
  bool active;
  const SoundEvent* sequence;
  byte length, index;
  unsigned long startedAt;
};
```

Funzionamento:

1. `emitStopSound()` carica la sequenza e avvia il primo evento.
2. Ad ogni giro del loop, `tickSound()` controlla se il tempo dell'evento corrente è scaduto.
3. Se sì, avanza al prossimo evento.

Niente `delay()` da nessuna parte. La sequenza di stop è:

```
{TONE_DOWN, 75}, {0, 150}, {TONE_DOWN, 75}, {0, 150}, {TONE_DOWN, 150}
```

(beep + pausa + beep + pausa + beep lungo).

Il mute: controlla `mute` globale. Se mute è ON, `emitSound()` è no-op.

---

11. Il relè e le macro PORTL

Questa parte è la più "vicina al ferro". Vale la pena spiegarla.

11.1 PORTL bit PL2

Il pin D47 della Mega è fisicamente collegato a `PORTL` bit 2 (PL2) del chip ATmega2560. Per accendere/spegnere il relè invece di usare `digitalWrite(47, HIGH)` (lento), usiamo:

```
#define RELAY_ON()  (PORTL |=  (1 << PL2))   // accendi bit
#define RELAY_OFF() (PORTL &= ~(1 << PL2))   // spegni bit
```

Vantaggio: velocità + utilizzabile dentro la ISR. `digitalWrite` chiama funzioni Arduino che non sono ISR-safe.

11.2 PORTL NON è in I/O range bassa

Su AVR i registri I/O 0x00-0x1F supportano `SBI` / `CBI` (1 istruzione, 2 cicli, **atomiche**).

PORTL su Mega 2560 è all'indirizzo 0x10B (extended I/O). Le SBI/CBI **non funzionano lì**. GCC genera:

```
lds r24, 0x10B    ; carica PORTL
andi r24, ~mask   ; clear bit
sts 0x10B, r24    ; scrivi
```

3 istruzioni, NON atomiche.

11.3 Perché è sicuro comunque

- **Dentro la ISR:** AVR disabilita gli interrupt globali (`SREG.I = 0`) quando entra in una ISR. Niente può interrompere. Sicuro.
- **Dal main:** nessun'altra ISR scrive PL2. L'unica ISR che scrive PORTL è quella di `tone()` (Timer2), e tocca solo PL0 (D49 = buzzer). Race teorica sui bit di PORTL diversi da PL2 = al massimo un glitch sul ciclo del buzzer (impercettibile). PL2 viene sempre scritto correttamente.

Quindi il pattern `PORTL &= ~(1<<PL2)` da main loop non è atomico ma **funzionalmente sicuro** per il relè.

11.4 Il "relay gate" `g_relayGate`

Quando un timer viene armato (`startCountdownTimer` , `startFocusTimer` , `startTestStripTimer`), si fa una **snapshot** del flag canale:

```
g_relayGate = channelsRelayActivated[channel - 1];
```

Quando la ISR vuole spegnere il relè a fine countdown:

```
if (g_relayGate) RELAY_OFF();
```

Usa la snapshot, non il flag corrente. Questo previene race condition se l'utente toggla il flag a metà del countdown. (In realtà non può, perché il keypad è disabilitato, ma è un pattern difensivo.)

---

12. Decisioni tecniche giustificate

12.1 Unità tempo = decisecondi

Tutto il firmware lavora in **decimi di secondo** (`timeSet` , `appoTimeSet` , `channelTime` , `nextStepTarget`).

Perché?

- L'utente vede 1 cifra decimale (es. `12.3` s). Decimi = precisione naturale.
- Range 0..9999 = 0..999.9 s. Sta in un `int16` (2 byte).
- Conversione ms ↔ decimi è banale: `decis * 100 = ms` .
- Evita float ovunque (lento su AVR).

12.2 Tutto in PROGMEM per le stringhe

`MODES_TEXT_*` , `FSTOP_STR_*` , `SEC_STR_*` sono in `PROGMEM` (flash), non in RAM.

Lettura via `strcpy_P(keyBuffer, ptr)` → copia da flash a RAM giusto prima di mostrarla.

Perché?

- RAM = 8 KB, ce ne avanza poca.
- Flash = 256 KB, ne avanza tanta.
- Le stringhe sono costanti, non c'è motivo di averle in RAM.

Risparmio post-refactoring: ~80 byte di RAM solo dalle stringhe.

12.3 `keyBuffer[17]` condiviso

Una sola variabile globale `char keyBuffer[17]` usata per tutto: format numerico, lettura PROGMEM, scratch.

Perché?

- Niente allocazione dinamica su AVR (`new` , `malloc` sconsigliati).
- Niente `String` class (~100 byte di overhead per istanza + frammentazione heap).
- 17 byte = 16 colonne LCD + null terminator. Sufficiente per qualsiasi stringa LCD.

Regola: usa `keyBuffer` per dati transitori, non per dati persistenti. Non assumere che il suo contenuto sopravviva dopo l'uso immediato.

12.4 No `delay()` nel loop path

Solo `delay(3000)` nello splash di setup. Altrove mai.

Perché?

- `delay()` blocca il loop. Durante quel tempo non si legge il main button, non si gira il countdown, non si fa nulla.
- I suoni a sequenza usano `SoundPlayer` (macchina a stati).
- Il countdown usa la ISR.
- L'unico uso accettabile è in `setup()` per il messaggio di benvenuto, dove non c'è ancora nulla da fare.

12.5 Niente `String` class, niente heap

Tutti i buffer sono static/global. Niente `new`, `malloc`, `String`.

Perché?

- AVR ha 8 KB di RAM. Lo heap frammenta in fretta.
- `String` class è notoriamente problematica per memory leak su AVR.
- `sprintf` + buffer statici fa lo stesso lavoro con zero overhead.

12.6 ISR a 1 kHz e non più alta

1 ms è la granularità del countdown. Si poteva fare 10 kHz, ma:

- Più ISR = più overhead. A 1 kHz l'ISR consuma ~5-10% del CPU.
- 1 ms di precisione è eccellente per un'esposizione fotografica.
- A 10 kHz si sarebbero rovinati altri timer del firmware Arduino (millis, tone).

12.7 ISR minimale, lavoro pesante in main

L'ISR fa solo:

- Decrementi/incrementi semplici.
- Confronti.
- Set di flag.
- Scrittura PORTL (1 bit).

NON fa:

- Chiamate a funzioni Arduino (digitalWrite, Serial.print, ecc.).
- Math floating point.
- Rendering LCD.
- Calcoli F-stop.

Tutto questo va nel main loop, alla cadenza di 100 ms. Mantiene l'ISR breve = jitter basso = countdown preciso.

12.8 Single Translation Unit

Già spiegato in §3.1. Riassunto: più ottimizzazione, niente prototipi/extern, niente linker headache. Prezzo: ordine include rigido.

---

13. Flussi concreti

13.1 Boot del firmware

```
1. main.cpp setup():
   - Serial.begin(9600)        [USB debug @ 9600 baud]
   - relè OFF (digitalWrite)
   - pin mode + pull-up main button
   - configKeypad (debounce 50 ms)
   - configLCD (init Waveshare, RGB rosso, custom chars)
   - config7SegmentsDisplay (SparkFun reset, brightness default)
   - showWelcomeMessage ("- R080 Timer - / - MEGA -")
   - delay(3000)                ← ok solo qui
   - loadSettings (EEPROM)
     se valido: brightness, mute, canale, channelTime[], channelsRelayActivated[] ripristinati
     se non valido: default
   - reset7SegmentsDisplay (azzera display)
   - resetLCD (pulisce LCD)
   - initializeOperations (crea i 3 oggetti Operation)
   - registerClickHandler (link encoder button → callback)
   - configureMainButton (debounce, click, press ms)
   - showSound, showChannel, showTime, showRel (UI iniziale)
   - setupTimer1 (parte il motore temporale 1 kHz)
   - Serial.println("=== R080 TIMER PRONTO ===")

2. main.cpp loop():
   - parte il ciclo infinito.
```

13.2 L'utente fa un'esposizione in COUNTDOWN

Scenario: canale A, 5.0 s, modalità SECONDS, precisione 1s.

```
T0: utente digita "5", "0" sul keypad
   evaluateKey RELEASED case '0':
     nCifra=0: newTimeSet = 5
   evaluateKey RELEASED case '0':
     nCifra=1: newTimeSet = 5 * 10 + 0 = 50
   timeSet = 50, appoTimeSet = 50, channelTime[0] = 50
   fstopState.setBase(50)
   showTime(50) → 7-seg mostra "0.050" → letto come "5.0"
   saveSettings() → EEPROM aggiornata

T1: utente preme main button (click)
   onMainButtonClick: isMainBtnPressed = !false = true

T2: prossimo giro di loop()
   mainButton.tick() processa il click
   manageMainButtonAction(): isMainBtnPressed=true → manageStarted()
   manageStarted() → operationStatus.current.startedAction()
   → countdown() (function pointer della modalità COUNTDOWN)
   countdown() → !isActionStarted=true → startCountdownTimer()

T3: startCountdownTimer()
   if (appoTimeSet <= 0) return   [guard: 50>0, ok]
   timerCurrentValue = 50
   relayStart(true) → canale A attivato → RELAY_ON() → lampada accesa
   isActionStarted = true
   showTime(50)
   ATOMIC: g_remainingMs = 5000, g_relayGate=1, g_modeActive=2

T3+1ms: ISR scatta
   g_tickMs++
   case 2: g_remainingMs=5000, decrementa a 4999
   g_100Counter incrementa

T3+100ms: ISR g_100Counter=100, alza g_tickFlag100ms
   main loop next: flag100=1
   onTick100ms() → mode==2 → legge g_remainingMs=4900 → mostra "4.9"

T3+1000ms: ISR g_remainingMs scende a 4000
   condizione (g_remainingMs % 1000 == 0) → alza g_metronomeFlag
   main loop next: flagMetro=1
   onMetronomeBeat() → emitSound(TONE_DOWN, 80) → beep 300 Hz × 80 ms

(stessa cosa a 3000, 2000, 1000 ms residui)

T3+5000ms: ISR g_remainingMs scende a 0
   if (g_relayGate) RELAY_OFF() → lampada spenta AL MS PRECISO
   alza g_endOfCountdown
   main loop next: flagEnd=1
   onCountdownEnd():
     isActionStarted = false, isMainBtnPressed = false
     ATOMIC: g_modeActive=0, g_remainingMs=0
     emitStopSound() → sequenza beep di stop
     timeSet = appoTimeSet = 50 [repeat last exposure]
     showTime(0)

Stato finale: utente vede "0.000" sul 7-seg per un attimo, poi il display mostra di nuovo "5.0" pr
```

13.3 L'utente fa un test strip continuo

Scenario: canale A, 5.0 s base, F-stop precisione 1/2 stop, CONTINOUS.

```
T0: utente imposta 5.0 s come sopra
T1: utente preme bottone encoder precisione → SECONDS diventa FSTOP
    fstopState.setBase(50) → base=50, offset=0
T2: utente preme main button → startTestStripTimer
    elapsedSnap = 0 → fresh start
    nextStepTarget = 50 (default = base)
    g_elapsedMs=0, g_nextStepTargetMs=5000, g_modeActive=3
    RELAY_ON()

T3..5s: ISR incrementa g_elapsedMs ogni ms
        main rendering ogni 100 ms mostra il count in salita

T2+5000ms: g_elapsedMs >= 5000 → alza g_nextStepReached
    main loop: flagStep=1
    onTestStripStep():
      timerCurrentDecis = 50
      emitSound(TONE_UP, 150) → beep step
      precisionValue = 2 (1/2 stop)
      FSTOP: multiplier = 2^(1/2) = 1.414
      newTarget = round(50 * 1.414) = 71 decis = 7.1 s
      g_nextStepTargetMs = 7100
      modalità CONTINOUS: NON spegne relè, NON ferma timer
    main continua a contare in salita

T2+7100ms: ISR alza g_nextStepReached
    onTestStripStep: beep, newTarget = round(71 * 1.414) = 100 decis = 10.0 s

T2+10000ms: beep, newTarget = round(100 * 1.414) = 141 decis = 14.1 s

(continua finché l'utente preme main button per fermare)

Tutti gli step: 5.0 → 7.1 → 10.0 → 14.1 → 19.9 → 28.2 → ...
```

13.4 L'utente fa un test strip step-by-step

Stesso scenario di sopra ma modalità STEP_BY_STEP (toggled con bottone encoder modalità).

```
T2+5000ms: step raggiunto
   onTestStripStep:
     beep
     newTarget = 71 decis
     modalità STEP_BY_STEP:
       RELAY_OFF()              ← lampada spenta
       isActionStarted = false
       isMainBtnPressed = false ← pulsante "resettato"
       showTime(71)             ← mostra prossimo target sul 7-seg
       g_modeActive = 0         ← timer fermo, g_elapsedMs CONGELATO a 5000

(utente ora vede sul display il prossimo target, sposta il cartoncino sulla striscia)

T?: utente ripreme main button
   isMainBtnPressed = true
   loop: manageStarted() → testStrip() → !isActionStarted=true → startTestStripTimer
   startTestStripTimer():
     elapsedSnap = 5000 ≠ 0 → resume mode, NON resetta nulla
     relayStart(true) → RELAY_ON
     g_relayGate = canale
     g_modeActive = 3

L'ISR riprende da g_elapsedMs=5000, target=7100. Continua fino al prossimo step.
```

13.5 L'utente cambia canale durante setup

```
canale A: timeSet=50, canaleTime[0]=50
utente preme B (RELEASED):
  switchToChannel(2):
    channelTime[0] = 50 (salva)
    channel = 2
    timeSet = channelTime[1] = 100 (es. canale B aveva 10.0 s)
    appoTimeSet = 100, nCifra = 0
    fstopState.setBase(100)
    showChannel() → LCD mostra "Ch:B"
    showTime(100) → 7-seg mostra "10.0"
    saveSettings() → EEPROM aggiornata (canale e tempi)
```

13.6 L'utente resetta un canale

```
canale corrente: A, timeSet=50
utente preme D (RELEASED):
  case 'D':
    channelTime[0] = 0
    fstopState.reset() → baseTime=0, stopOffset=0
    resetTime():
      reset7SegmentsDisplay() → mostra "0.000"
      timeSet = 0
      nCifra = 0
      appoTimeSet = 0
    saveSettings() → EEPROM aggiornata
```

13.7 L'utente toggla il mute (hold *)

```
utente tiene premuto *
  evaluateKey() entra in case HOLD:
    case '*':
      mute era false → mute = true → showMute() (icona nota cancellata)
      saveSettings()
    (cade in PRESSED/IDLE: return)

utente tiene premuto * di nuovo
  evaluateKey() case HOLD case '*':
    mute era true → mute = false → emitSound(TONE_UP, 200) → beep conferma
    showSound() (icona nota ripristinata)
    saveSettings()
```

---

14. Cosa NON si può fare (vincoli AVR)

1. **Niente `new` / `malloc` / `delete`** — heap su AVR è instabile.
2. **Niente `String` class** — frammentazione + overhead.
3. **Niente `std::` libraries pesanti** (vector, map, function) — non c'è la STL su AVR.
4. **Niente `float` nelle ISR** — la lib math FP non è ISR-safe.
5. **Niente `Serial.print` nelle ISR** — Serial è async e usa buffer.
6. **Niente `delay()` nelle ISR** — `delay()` aspetta `millis()` che è incrementato da un'altra ISR.
7. **Niente accesso multi-byte non protetto** a variabili volatili dal main — race con ISR.

8. **Niente exception / RTTI** — il compilatore AVR le supporta ma costano caro in flash.
9. **Niente threading** — AVR è single-core. Pseudo-parallelismo solo via ISR.

---

15. Cose che NON devi cambiare (a meno di sapere cosa fai)

Dalle "trincee" dei refactoring precedenti:

1. `enum ModalityType { CONTINOUS, ... }` — scritto male di proposito. Cambiarlo rompe ovunque sia referenziato.
2. `HOLD:` **senza** `break` in `evaluateKey` — intenzionale (vedi §9.2).
3. `MATRIX_KEYS[COLS_N][ROWS_N]` **+** `makeKeymap(ROWS_PIN ...)` — i due swap si cancellano. Cambiare uno solo rompe il keypad.
4. **L'ordine degli include in** `main.cpp` — `timer_isr.h` deve precedere `base.h`.
5. **Le macro** `RELAY_ON()` **/** `RELAY_OFF()` **con PORTL** — non tornare a `digitalWrite`, perché la ISR ha bisogno di un flip in 1 istruzione macchina.
6. **Niente secondo** `.cpp` **che includa** `variables.h` — multiple definition. Solo `main.cpp` lo include.

---

16. Bug fixati nei refactoring precedenti (NON regredire)

1. **Race condition countdown** — c'erano 2 timer concorrenti, ora solo Timer1 ISR.
2. `appoTimeSet / 10 * 10` — lossy, ora copia diretta.
3. `delay()` **nei suoni** — sostituito con macchina a stati.
4. **Doppie scritture LCD** — c'erano 5 singleton manager che facevano refresh duplicati, rimossi tutti.
5. **EEPROM non persistente** — aggiunto magic 0xA5 + load/save.
6. **Repeat-last-exposure** — `timeSet = appoTimeSet` a fine countdown.
7. **F-stop con conteggio stop e navigazione** `*` **/** `#` — implementato da zero.
8. `channelsRelayActivated[channel]` — off-by-one, corretto a `[channel-1]`.
9. **Countdown con tempo 0** — guard `if (appoTimeSet <= 0) return` (2026-05-24).
10. **Codice morto** `setErrlet` **/** `isInError` — rimosso (2026-05-24).

---

17. Budget RAM/Flash attuale

```
RAM:   1463 / 8192 byte  (17.9%)
Flash: 24566 / 253952 byte (9.7%)
```

Più dell'80% di RAM disponibile e oltre 200 KB di flash per future espansioni. Ottimo margine.

Le aree di RAM più "calorose":

- `keyBuffer[17]` — 17 byte (riusato).
- `note[8]`, `spina[8]` — 16 byte custom chars.
- 3 oggetti `Operation` + 1 `OperationStatus` — ~50 byte.
- Variabili `volatile` ISR-shared — ~25 byte.

- Oggetti hardware (`Keypad`, `Encoder`, `OneButton`, ecc.) — ~200 byte.
- Stack frame del main loop e ISR — fluttuante.

---

18. Glossario rapido

- **ISR** — Interrupt Service Routine, funzione chiamata dall'hardware.
- **PROGMEM** — area flash, sola lettura, accesso via `pgm_read_*` / `strcpy_P`.
- **SBI/CBI** — istruzioni AVR per set/clear di un bit in I/O bassa (1 ciclo).
- **PORTL/PL2** — registro I/O del chip per i pin del Mega; PL2 = bit 2 = D47.
- **ATOMIC_BLOCK** — blocco di codice con interrupt disabilitati (AVR-libc).
- **TU** — Translation Unit, un'unità di compilazione (un `.cpp` con i suoi include).
- **decis** — decimi di secondo, l'unità di tempo interna.
- **F-stop** — unità fotografica di esposizione, raddoppio/dimezzamento luce.

---

19. File di riferimento

- `platformio.ini` — config build, dipendenze, board.
- `CLAUDE.md` — guida operativa per AI assistant (overlap con questo doc, ma orientato a chi modifica).
- `DOCS.md` / `DOCS-IT.md` — manuale utente (cosa fa il timer).
- `README.md` — installazione, pinout sintetico.
- `src/main.cpp` — entry point.
- `include/functions/*.h` — moduli funzionali (vedi §3.2 per ordine).

Fine documento.